CONVERSATION RESEARCH DIGEST

CUA 科研与自主编程体系

本次对话完整总结

从十项前沿研究，到 DeepSeek Harness + DeepSeek API 的 Linux CUA 落地路线

核心主题：文献审查·系统改造·边写边测·证据驱动修复·独立验收

研究设计与谈话纪要合集（非已完成实验报告）

资料日期：2026 年 11 月 9 日

文档类型：可复制、可检索的中文文字 PDF

目录

执行摘要：这次讨论最后真正确定了什么 **2**

第一部分：对话进程与观点演变 **3**
1.1 起点：要求十项高质量 CUA 前沿研究 . . . 3
1.2 第二步：用户提交十篇多智能体文献综述，请求评价 . . . 3
1.3 第三步：从文献导读转成研究项目和实施计划 . . . 4
1.4 第四步：用户补充真实技术底座，科研问题发生实质变化 . . . 4
1.5 第五步：用户要求“史诗长文”，系统性论证改造价值 . . . 5
1.6 第六步：研究主线明确转向“高质量编程、边写边测” . . . 5

第二部分：最终确认的系统定位与设计原则 **5**
2.1 目标系统的分层职责 . . . 5
2.2 不修改官方 Harness 核心的约束 . . . 6
2.3 GUI 和终端的合理分工 . . . 6
2.4 模型能力与系统能力的边界 . . . 6

第三部分：“边写边测”的最终详细定义 **6**
3.1 从口号变成完整闭环 . . . 6
3.2 代码库理解不等于整仓塞进上下文 . . . 7
3.3 小步增量修改的工作契约 . . . 7
3.4 T0—T6 七层测试体系 . . . 7
3.5 错误诊断：区分症状、假设、验证和根因 . . . 8
3.6 修复状态机与停止条件 . . . 8
3.7 防止“改测试让程序变绿” . . . 9
3.8 质量目标不仅是“能跑” . . . 9

第四部分：独立验证、状态持久化与执行安全 **9**
4.1 可信完成状态 . . . 9
4.2 状态记录与中断恢复 . . . 9
4.3 权限与不可逆操作边界 . . . 9

第五部分：多智能体在最终方案中的位置 **10**
5.1 多 Agent 是研究变量，不是项目先决条件 . . . 10
5.2 四种最重要的协作基线 . . . 10
5.3 共享 Linux 桌面的具体冲突 . . . 10
5.4 更谨慎的增量引入策略 . . . 10

第六部分：统一科研实验与结果判定 **10**
6.1 从早期四组到后期六组的变化 . . . 10
6.2 实验任务与规模 . . . 11
6.3 核心指标 . . . 11
6.4 能够推翻方案的失败判据 . . . 11

第七部分：文献来源、准确性与证据分级 **11**

7.1 用户提交的十项多智能体基础材料 11
7.2 软件工程与 CUA 方向的补充资料 12

第八部分：交付计划与明确的阶段放行门槛 **12**
8.1 推荐的执行顺序 12
8.2 讨论中反复推荐的首个验收任务 13
8.3 交给开发 Agent 的执行纪律 13

第九部分：当前成果、尚未完成工作与风险 **13**
9.1 对话中已经完成的“文稿类成果” 13
9.2 没有证据表明已经完成的工作 14
9.3 重大风险与对应措施 14

第十部分：最终共识与最小下一步行动 **14**
10.1 可以概括为一句话的项目方向 14
10.2 首轮行动清单 14
10.3 完成后应提交的证据包 15

附录 A：本次对话涉及的公开文献与项目入口 **15**

附录 B：本次对话中关键概念的统一解释 **15**

阅读说明 本文整理的是本次对话中已经表达的目标、技术观点、研究提案、争议、更正和下一步方案，不是新的实验论文。所有尚未开展的实验、未核实的框架接口和推测性性能收益，均按“拟议”“待核实”处理。文中保留重要的技术细节，但不逐字转录所有聊天文字。资料时间范围为 **2026 年 11 月 9 日的本次会话**。

执行摘要：这次讨论最后真正确定了什么

本次对话最初以“写十篇真实、不谄媚、具有科研价值的 CUA 前沿研究”为起点，随后引入 CAMEL、AutoGen、MetaGPT、ChatDev、Reflexion、Multi-Agent Debate、GPTSwarm、AgentVerse、AgentBench 和 OSWorld 这十个多智能体或评测方向；在对这些文献的准确性与适用性作审查后，讨论从“蜂群式多智能体系统”逐渐收敛到一个更具体、也更重要的工程目标：

以 DeepSeek Harness 为运行底座，以 DeepSeek API 为主要推理核心，通过不修改官方核心的外围扩展，在 Linux 环境中建设能够理解真实代码库、小步写代码、持续测试、分析根因、自动修复、回归验证并交付可运行软件的 CUA。

对话形成的主要共识是：

1. **先证明单 Agent 能交付真实软件，再研究多 Agent 是否有额外收益**。“蜂群”不是必须完成的终点。
2. **Harness 是运行与编排底座，API 是推理端，执行与验证能力需要单独建设**。增加模块不能凭空提高模型本体推理能力。
3. **“边写边测”是自主软件工程闭环，而不是在写完代码后执行一次测试**。必须形成“理解—定位—修改—测试—诊断—修复—回归—验收”的可追踪机制。
4. **真实证据决定完成状态**。文件已生成、工具返回成功、测试命令退出码为零、另一个模型表示认可，都不自动等于任务成功。
5. **科研提案与已完成成果必须严格区分**。本次讨论已产生文献评价、系统方案、实验设计和研

究文档；没有证据表明相关 Linux CUA 原型、蜂群系统或所有实验已在真机完成。

6. **新增功能必须接受消融与对照实验**。规划、记忆、反思、蜂群如果没有净收益，就应简化、限制或移除。

本次交流最重要的转折，是用户明确提出：**后续工作应实现高质量编程能力，特别是“边写边测”**；随后对第一版总结批评“太敷衍”，促使技术方案升级为包含仓库理解、修改契约、测试分层、错误状态机、恢复机制、实验与验收的详细实施规范。

第一部分：对话进程与观点演变

1.1 起点：要求十项高质量 CUA 前沿研究

用户首先提出写十篇“高质量的 CUA 前沿研究”，特别强调“真实”“不谄媚”和科学研究价值。助手没有把“伟大”当作既成事实，而是组织了十个可证伪的研究主题：

编号	研究主题	核心科学问题
01	自然语言意图到可验证任务契约	Agent 怎样理解真正的交付目标，而不是机械执行字面步骤?
02	Linux GUI 多模态定位	截图、辅助功能树、窗口状态怎样融合才真正减少误操作?
03	反馈驱动重规划	失败后何时重试、局部修复、整体重规划或停止?
04	自主软件工程	能否真正做到读仓库、写代码、运行、修复与交付?
05	独立验收与可信交付	怎样区别“声称完成”和“独立验证通过”?
06	长任务持久化与幂等	崩溃后如何恢复且不重复外部副作用?
07	权限、危险动作与撤销	怎样在可控范围内操作真实计算机?
08	诚实性与不确定性校准	什么情况下应说“未知/无法验证”?
09	蜂群式多智能体协作	何时协作有收益，何时因冲突与成本而退化?
10	经验检索与抗退化持续改进	不更新模型参数，经验库能否稳定改善系统任务表现?

这十项的共同研究纪律是：真实基线、对照组、消融实验、失败案例、成本与时延、独立验收、版本固定和可复现记录。不能把研究方案或未经运行的代码写成实证成果。

1.2 第二步：用户提交十篇多智能体文献综述，请求评价

用户提供了一份题为《LLM 多智能体与集群协同体系：核心文献全景白皮书》的内容，主要覆盖十项历史研究或基准：CAMEL、AutoGen、MetaGPT、ChatDev、Reflexion、Multi-Agent Debate、GPTSwarm、AgentVerse、AgentBench、OSWorld。

助手的评价重点并非否定这些研究，而是指出：**选题较有代表性，但它们并非都直接研究 Linux CUA，更不能证明“多个 Agent 一定更强”**。先前的主观评分依次为“研究选题 7/10、机制解释 5/10、学术严谨性 3/10、CUA 工程落地价值 3/10”，仅针对用户提交的综述文本，并非对原论文本身的权威评分。

指出的关键问题包括：

- **论文题名不准确**。CAMEL 和 AgentVerse 的英文题名需按原文核对，不应靠二手材料直接引用。
- **结论过度绝对化**。如“MetaGPT 彻底根治幻觉”“AutoGen 解决代码不能验证的问题”“多 Agent Debate 共识必然正确”等表述缺乏普遍性证明。
- **机制被过度简化**。用一般状态方程描绘 CAMEL 不等于重现作者的算法推导；GPTSwarm 的可优化图不等于已证明在真实 CUA 中可以无代价在线实时增删拓扑。
- **基准与架构混淆**。AgentBench、OSWorld 是重要评价框架或任务环境，但不能据其成绩单独推断“多智能体协同优势”。
- **缺少可复现细节**。原综述没有系统列出原论文实际实验、统计结果、消融、代码版本、失败条件、许可证与适配边界。

因此，形成了“**文献材料应先查核、分层归档，再转为候选工程机制**”的共同方向。

1.3 第三步：从文献导读转成研究项目和实施计划

用户认可上述评价后，助手提出正式研究目标：**Linux CUA 多智能体协同科研项目**。其起初的安排有四个阶段：

1. 逐篇审查原始论文与公开实现；
2. 建立区分证据、推断、假设的知识库；
3. 用真实任务做单 Agent 与多 Agent 对照实验；
4. 根据实验结果选择工程实现，不先决定必须采用蜂群。

随后用户要求把“论文和计划一起写出来”。在此前回复中，助手表示已提供一份名为《Linux CUA 十项科研论文与完整实施计划》的 PDF 和 Word；本次整理时检查到对应资料文件，原 PDF 为 **16 页**，属于十项研究论文提案与阶段实施计划合集，不是十篇已完成投稿论文。

该旧文档规划了四组核心实验：A 单智能体、B 固定角色串行流水线、C 共享工作区并行蜂群、D 隔离工作区并行协作；另提出 **120 项任务 × 4 组 × 3 次重复 = 1,440 次拟议运行**。这个数字只是一种资源预算与实验设计占位，尚未证明已经执行，也必须在正式实验前通过功效分析调整。

早期计划中的特别约束包括：同模型、同权限、同验收器、可比较资源预算；单桌面会话的键鼠写入要有仲裁；必须区分执行日志、模型自述和独立验收证据；首先交付可真正打开、保存、关闭后重新打开的本地软件。

1.4 第四步：用户补充真实技术底座，科研问题发生实质变化

用户随后明确：

> 底座采用 **DeepSeek Harness**；思考端采用 **DeepSeek API**；希望在此基础上实施前述改装，并询问“真的有意义么”。

助手的核心判断是“**有条件地有意义**”：它可以提升系统层面的可执行性、可观察性、故障恢复、验证和任务管理，但不能保证提高模型本体的推理上限，也不能确保十个模块全部装入就更优秀。

对话由此转向实际架构：DeepSeek API 为主推理器；Harness 为运行与工具协调层；外围服务负责仓库检查、文件修改、命令运行、桌面操作、测试、验收、状态和未来协作。**不修改官方 Harness 核心**被反复确立为重要约束，优先利用经过实际核实的官方扩展机制；MCP、Skills、插件等能否接入，必须检查当前安装版本与真实 Schema，不能从概念图直接假定可用。

助手也强调应当区分两类失败：

- **模型推理瓶颈：**信息充分时仍不能正确设计或诊断，可能需要其他模型、检索、外部求解工具或缩小任务。
- **系统工程瓶颈：**工具不可达、错误反馈不足、GUI 失焦、未执行测试、上下文状态丢失等，这些才是改造执行层最直接的价值空间。

1.5 第五步：用户要求“史诗长文”，系统性论证改造价值

助手随后给出一篇大型研究与工程战略长文，主题是从语言模型走向自主软件工程实体。内容强调：架构分层、模型与系统能力的区别、工具反馈、程序交付、错误恢复、经验库、蜂群边界、Linux GUI/终端混合操作、模型调用预算及科学实验。

长文特别提出一种“证据驱动的能力跃迁”观：新增能力必须使真实任务在相同约束下更容易通过验收；不能凭角色数量、提示词长度或思考轮数衡量所谓智能升级。

1.6 第六步：研究主线明确转向“高质量编程、边写边测”

用户进一步提出：**后续工作希望实现高质量编程能力，要求 Agent 边写边测**。

助手最初给了较简短的路线：检查仓库、任务规划、小步编写、静态检查、运行、错误修复、回归、交付；并建议将编程闭环置于蜂群之前。

用户明确反馈“**重写，这太敷衍**”。随后的重写成为本次对话最细致的实施方案：将边写边测扩展为包括仓库上下文选择、增量修改契约、七层测试、根因诊断、修复状态机、防止测试作弊、状态持久化、工程质量评价、调用成本优化、CUA 混合执行和阶段验收的完整研究规范。

这次反馈在项目方法论上的意义是：**用户需要的是可交给开发 Agent 实施并验收的工程文档，而不是口号、阶段标签或泛泛而谈的流程**。

第二部分：最终确认的系统定位与设计原则

2.1 目标系统的分层职责

层	主要职责	不能直接推断的能力
DeepSeek API	需求理解、计划、代码生成、错误分析、决策	不自动拥有真实桌面与文件权限，也不保证推理正确
DeepSeek Harness	模型/工具交互、上下文组织、运行循环，使用真实支持的扩展点	不应假定任意生命周期钩子、插件或持久化接口必然存在
工程执行服务	文件读取、受控编辑、命令运行、测试执行、GUI/浏览器适配	工具调用成功不代表用户目标已完成
证据与验收层	保存日志、差异、测试报告、真实产物、重启验证	验收器可能有缺陷，必须有独立性和版本管理

层	主要职责	不能直接推断的能力
状态与恢复层	任务进展、检查点、外部副作用标记、崩溃恢复	不能普遍保证所有外部动作“恰好一次”
经验与协作层	经验证技能检索、多 Agent 任务拆分与合并	记忆变大和 Agent 变多不保证成功率提高

理想执行路径是：

用户目标 → 任务与验收契约 → DeepSeek API / Harness → 选用适当工具 → 真实执行 → 环境证据 → 判断完成、失败或未知 → 下一步决策。

2.2 不修改官方 Harness 核心的约束

本次对话反复强调以下原则：

- 先检查实际安装的包名、版本、发布来源、源码 SHA、扩展入口与工具 Schema；
- 优先采取官方允许的插件、Skills、MCP 或配置扩展；具体可用性以当前版本证据为准；
- 如果扩展点不足，先考虑外围服务适配，而不是凭空生成不可应用的 diff；
- 保留基线可重放能力，使上游更新、失败归因和对照实验可控；
- 遇到做不到的接口能力，明确写“当前不支持”或“未核实”，不能为了架构完整而虚构。

2.3 GUI 和终端的合理分工

Linux CUA 不应把所有动作强制变成鼠标操作。文件读取、代码修改、构建、单元测试等可以优先使用可验证的文件系统、终端或稳定 API；必须检查真实软件界面与用户操作路径时再使用 GUI。

需要特别记录 Linux 发行版、桌面环境、X11/Wayland、缩放、截图与辅助功能接口、窗口焦点和执行权限。某一 X11 工具成功，不代表全部 Wayland 桌面都支持相同操作。对共享 GUI，多个 Agent 同时拥有键鼠写入权将引发资源冲突；应研究单写入租约或串行仲裁。

2.4 模型能力与系统能力的边界

讨论接受的基本判断：**模型参数不变，系统能力仍有可能通过工具、反馈、检索和验证得到改善，但提升幅度与任务范围需实验。**

不能声称 Harness 可以把不会解决的问题“变聪明”。应当比较固定模型在不同执行系统下的真实成功率，并分别记录模型错误、环境错误、执行错误、验证缺失和预算限制。

第三部分：“边写边测”的最终详细定义

3.1 从口号变成完整闭环

讨论最终采用的主流程是：

理解需求 → 审计环境 → 理解仓库 → 查找相关代码与资料 → 设定工作单元 → 修改代码 → 检查差异 → 运行相应测试 → 分析真实失败 → 最小范围修复 → 回归验证 → 独立交付。

这不是要求每次都机械地完成全部步骤，而是根据工作风险与证据密度选择适当阶段。测试失败需要回到诊断，而不是无限重写文件。

3.2 代码库理解不等于整仓塞进上下文

重写方案提出四层上下文管理：

1. **结构层：**识别项目语言、构建方式、依赖、入口、测试目录和配置。
2. **定位层：**搜索相关函数、类、调用方、测试文件与错误位置。
3. **依赖层：**理解接口契约、调用关系、生成文件与跨模块约束。
4. **最小有效上下文层：**优先读取完成当前修改必需的真实代码、配置和错误证据。

曾提出一个候选文件优先级函数，把语义相关性、符号匹配、依赖关系和错误关联纳入评分；它属于**拟议方法**，权重尚未通过实验确定。

研究目标不是尽量增加 Token，而是在有限预算中减少错误定位与无关代码修改。

3.3 小步增量修改的工作契约

每个工作单元至少明确：

字段	作用
Goal	当前要解决的具体问题
Scope	允许修改的文件、函数和接口
Preconditions	执行前应成立的环境/代码条件
Baseline	修改前工作区版本和既有测试状态
Patch	实际差异，优先可审查的局部修改
Tests	本次修改需要执行的相关测试
Expected result	修改后的可观察行为
Rollback	修改失败时的可控恢复办法

严格禁止对尚未读取的文件盲目全文覆盖；如文件已被其他进程修改，必须重新读取并核对版本。用户原有未提交修改不能被 Agent 为了回到绿色状态而随意丢弃。

3.4 T0—T6 七层测试体系

层级	名称	主要检查目标
T0	修改有效性与差异检查	文件真的变更、范围符合工作契约
T1	语法/编译检查	发现语法及基础构建错误
T2	静态检查	类型、规范、静态缺陷与潜在风险
T3	单元测试	单函数、类、模块逻辑
T4	集成测试	模块之间的真实接口行为
T5	端到端测试	用户可见的完整业务链路
T6	交付验收	构建、运行、关闭、重启、读回与目标功能

关键澄清：边写边测不是每改一行就跑一次完整 E2E。应根据修改影响范围进行自适应测试选择：局部函数偏向单测，公共接口扩大到集成测试，数据库与认证模块需要更严格的回归，最终

交付执行真实用户路径验收。

测试执行器要返回真实工作目录、执行命令、环境标识、退出码、stdout、stderr、用时、运行测试数量、通过/失败/跳过数、日志位置、唯一执行 ID。退出码为 0 也不必然证明所有预期测试被发现和运行；零测试或错误测试配置应专门识别。

3.5 错误诊断：区分症状、假设、验证和根因

讨论指出，遇到 `ModuleNotFoundError` 不能总是直接安装依赖，因为真实原因可能是虚拟环境、导入路径、打包配置或版本问题。候选错误分类包括：

- 语法和类型错误；
- 依赖、环境与构建错误；
- 业务逻辑与集成错误；
- GUI、焦点、权限和工具协议错误；
- 并发/资源竞争与偶发错误；
- 测试设计本身存在缺陷。

合理诊断顺序为：收集原始报错 → 找到可信失败位置 → 检查近期差异 → 提出有限候选原因 → 设计最小复现实验 → 根据证据选择修复 → 重新跑失败测试和回归。不能因为语言模型输出了一个自信解释，就直接当作根因。

3.6 修复状态机与停止条件

建议的研究状态机：

```
TEST_FAILED
  ↓
COLLECT_EVIDENCE
  ↓
CLASSIFY_FAILURE
  ↓
GENERATE_HYPOTHESES
  ↓
SELECT_HYPOTHESIS
  ↓
MINIMAL_PATCH
  ↓
RUN_TARGETED_TESTS
  ├── 失败 → 重新诊断（不得无意义地重复同一补丁）
  └── 通过 → RUN_REGRESSION
                ├── 失败 → 修复或回滚
                └── 通过 → VERIFIED
```

还必须存在 `BLOCKED`、`BUDGET_EXCEEDED`、`ENVIRONMENT_UNAVAILABLE`、`UNSAFE_TO_CONTINUE` 等停止状态。若多次修复未产生进展，应重新检查定位、上下文、根因、回滚和模型能力，而不是无限生成新补丁。

对修复候选可以研究“预计收益 − 执行成本 − 回归风险”的排序思想；其中概率估计尚未经标定，早期优先规则与证据，不得假装模型给出的数字已经是可靠概率。

3.7 防止“改测试让程序变绿”

这是讨论中重要的反作弊原则：开发 Agent 编写的辅助测试可以帮助定位问题，但最终验收不能完全由它修改和控制。原项目回归测试、任务契约和独立验收测试应保留完整变更记录。确需修改测试时，必须有需求变更或测试错误的证据；不得通过删断言、改预期值或跳过测试伪造成功。

3.8 质量目标不仅是“能跑”

后来重写的方案提出六个代码质量维度：**功能正确性、可维护性、可靠性、性能、安全性、可复现性**。每一维应使用独立可解释的证据，不宜随意混成一个虚构的“代码质量百分比”。高质量代码还要求避免无关文件修改、接口破坏、隐式依赖、回归错误和不可复现构建。

第四部分：独立验证、状态持久化与执行安全

4.1 可信完成状态

整个对话始终强调如下区分：

状态	实际含义	能否直接算成功
Planned	已产生计划	否
Executed	已发出或执行某动作	否
Produced	已生成文件或产物	否
Verified	独立验收通过	是
Failed	有明确失败证据	否
Unknown / Pending	缺少证据或无法执行验收	否

“模型说完成”“工具返回 OK”“测试退出码零”都只是部分证据。针对真实软件，还应独立构建、启动、运行、关闭、重新启动以及检查持久化数据读回。

4.2 状态记录与中断恢复

重写方案建议任务状态至少包含任务 ID、目标、验收条件、当前阶段、工作区版本、当前修改、最后可信检查点、测试证据、未解决错误、调用预算、副作用状态与最终状态。**聊天记录不能替代真实执行状态**。

恢复时应读取日志并核验真实环境，确认哪些动作已生效、哪些可以安全重试、哪些处于不确定状态。外部服务可能已执行成功但回执丢失；没有幂等键或可查询状态时，不能宣称任意 GUI 点击都能“恰好一次”。对于无法确认的不可逆操作，应暂停并要求确认。

4.3 权限与不可逆操作边界

对危险动作如删除重要文件、发消息、购买、提权、改变系统设置与不可逆提交，应遵循既有授权和确认规则。测试、模拟和隔离实验可用于评估机制，但不能用来绕过真机权限。工程上尽量使用最小权限、独立工作区、版本快照和可补偿操作；不能把本地回滚等同于撤销已经发生的外部副作用。

第五部分：多智能体在最终方案中的位置

5.1 多 Agent 是研究变量，不是项目先决条件

在文献评价阶段就确立：多 Agent 可能通过并行工作改善复杂任务，也可能因通信、重复阅读、合并冲突、共享错误假设而变差。因此最终开发顺序应先建立强单 Agent 基线，再比较多角色协作。

5.2 四种最重要的协作基线

代号	架构	核心问题
A	单 Agent 完整执行	单体系统能达到怎样的可靠交付水平?
B	固定角色串行流水线	角色分工是否弥补了交接开销?
C	共享工作区并行蜂群	并发效率能否抵消共享状态冲突?
D	隔离工作区并行协作	隔离与合并能否改善总成功率和成本?

公平比较要求固定基础模型版本、推理参数、工具能力、权限和验收器；对多 Agent 必须统计**全团队** Token、时间和协调成本，而不是只计一个角色。单 Agent 优于蜂群也是有效科研结论。

5.3 共享 Linux 桌面的具体冲突

多 Agent 并发生成文本相对容易，但同时控制一个桌面会话的键盘、鼠标、窗口焦点、剪贴板和同一代码文件，容易出现错误输入与覆盖。对话明确提出资源租约、GUI 单写入者、工作区隔离、代码合并和独立验收等候选机制；这些都属于**待实验的工程设计**，并非已验证的最终架构。

5.4 更谨慎的增量引入策略

后来的编程研究方案建议先比较三种简化角色：单 Agent；开发 Agent + 独立验证 Agent；开发、测试、协调三角色。只有证明这些简化协作在真实任务中有净收益，才考虑更复杂的动态图或大规模蜂群。

第六部分：统一科研实验与结果判定

6.1 从早期四组到后期六组的变化

早期项目研究**单/多 Agent 拓扑**，核心为 A/B/C/D 四组。用户将目标明确改为“高质量编程边写边测”后，助手又提出更细的**累积式编程能力**分组：

- A：当前真实可运行的 Harness + DeepSeek API 基线；
- B：加入完整工具反馈与验证；
- C：加入小步增量修改；
- D：加入连续测试、故障诊断和自动修复；
- E：加入检查点和长任务恢复；
- F：加入多 Agent 协作。

两套分组不矛盾，但研究问题不同。先用能力分组证明“边写边测”的价值，再在一个稳定版本上用四拓扑实验研究蜂群是否值得做。不能把两套结果混成同一组性能声明。

6.2 实验任务与规模

讨论过的候选任务覆盖：文件与文档、Linux 工具、真实代码库修复、Web/API 开发、GUI 与终端混合任务、多文件开发、长任务与故障注入。早期建议 30—50 个探索任务；旧 PDF 另有 120 任务 ×4 组 ×3 重复的占位计划。这些数字都属于**拟议规模**，不是已完成实验，更不是无需功效分析的最终样本量。

对照实验还应使用固定环境快照、版本锁定、任务预定义、隐藏验收、足够重复、随机化或配对、置信区间与失败归因。测试任务与开发调参任务需分开，防止泄漏。

6.3 核心指标

主指标：

$$R_{\text{verified}} = \frac{N_{\text{verified}}}{N_{\text{assigned}}}$$

辅助指标包括：错误成功声明率、自动修复成功率、回归缺陷率、无效迭代率、故障恢复成功率、未经授权动作率、墙钟时间、Token/费用、协调成本与人工介入率。

针对协作，建议估计：

$$\Delta R = R_{\text{multi}} - R_{\text{single}}$$

但单次差值不足以证明普遍优势，应考虑任务配对、重复运行、效应大小与区间估计。多 Agent 若只略增成功率而显著增加成本，可能不值得保留。

6.4 能够推翻方案的失败判据

讨论形成的主要否证原则：

- 动态任务契约若不能改善独立验收或造成需求误改，应限制使用；
- 多模态 GUI 融合若比简明定位更不稳定，应拆分场景；
- 局部修复若持续错误或成本过高，应回到全局诊断或回滚；
- 自我反思若仅增加调用不增加正确性，应删除；
- 记忆系统若造成负迁移，应引入版本与过期策略，必要时关闭；
- 蜂群若在等预算下不如单 Agent，不应为了概念完整而保留；
- 自动恢复若重复不可逆副作用，则其可靠性主张不成立。

第七部分：文献来源、准确性与证据分级

7.1 用户提交的十项多智能体基础材料

文献/框架	本次对话提炼的研究价值	必须防止的误用
CAMEL	角色与对话任务推进	不能宣称已解决长任务工程执行
AutoGen	可编排的工具/Agent 协作	不能将某种执行配置等同通用行业标准

文献/框架	本次对话提炼的研究价值	必须防止的误用
MetaGPT	SOP 与结构化交付物	SOP 不能彻底消除幻觉与质量缺陷
ChatDev	虚拟软件开发流程	评审对话不能替代真实程序运行
Reflexion	反馈驱动语言反思	提示词记忆不是模型参数学习
Multi-Agent Debate	独立观点和相互辩论	共识与多数表决不等于事实
GPTSwarm	可优化协作图结构	不可直接声称已证明真实桌面在线动态重构
AgentVerse	任务组织与候选角色招募	角色招募不等于真实工具执行能力
AgentBench	Agent 环境评测	不专门证明多 Agent 协作
OSWorld	真实桌面任务基准	不应以基准存在来推出蜂群优势

讨论中曾出现关于 AutoGen 后续维护状态、OSWorld 后续版本的提醒。这类**会随时间变化的事实**在本次总结中不作未经重新查询的最新断言。正式选择依赖与评测版本前，必须再检查官方仓库、论文版本和迁移文档。

7.2 软件工程与 CUA 方向的补充资料

讨论还强调 SWE-agent、SWE-bench、OpenHands、Aider、BrowserGym、Temporal、Microsoft UFO、MCP 等方向：它们在真实代码修复、工具接口、执行环境、评测、持久化和桌面操作方面可能比纯语言蜂群论文更直接地支持本项目。

科研知识库的证据等级建议为：

1. **原文证据：**论文作者实际提出并测量了什么；
2. **公开代码证据：**对应版本源码真的实现了什么；
3. **工程推断：**根据资料推导的可尝试方案；
4. **待验证假设：**本项目计划通过实验验证的机制；
5. **未核实：**尚缺乏可靠资料或真机证据。

不能将以上等级混写为“已被证明”。

第八部分：交付计划与明确的阶段放行门槛

8.1 推荐的执行顺序

阶段	工作内容	放行条件
0	当前 Harness、DeepSeek API、Linux 工具和工作区审计	有版本、SHA、可用接口与真实工具调用证据
1	单 Agent 首个最小真实产品	能创建、启动、实际操作、保存、关闭、重新打开并读回
2	统一动作与测试证据	能区分成功、失败、未知并定位原始日志

阶段	工作内容	放行条件
3	仓库理解、增量编码、测试选择、自动诊断与回归	有真实修复案例及对照实验
4	长任务检查点与崩溃恢复	故障注入下无未确认不可逆动作自动重复
5	经验证经验与技能复用	相似任务收益可测，旧任务不明显退化
6	单/多 Agent 协作比较	等预算下有可复验增益才默认启用
7	形成研究报告与知识库	文献、复现实验、负面结果和限制齐全

这里的排序取决于客观放行条件，而不应因日历或开发文档很长就自动进入下一阶段。已有授权、密钥与操作确认边界照旧；首个产品交付前不应把精力转移到与当前成果无直接关系的宏大架构建设。

8.2 讨论中反复推荐的首个验收任务

首个产品可以是**小型 Linux 本地任务管理程序**，或一个具有输入框、保存按钮与本地持久化的小型 Web 应用。重要的不是应用名称，而是它具有可独立验证的真实闭环。

最低交付要求：创建项目、安装依赖、实现基础业务、运行静态/单元/集成检查、真实启动、完成用户交互、将数据持久化、关闭、重新启动、读回并验证原有数据。应保留代码差异、工具日志、测试报告和实际交付物。任何 Mock 通过、代码片段生成或模型声称成功都不能替代真机验收。真机条件不足则记录 Pending。

8.3 交给开发 Agent 的执行纪律

1. **先读环境再作决定：**版本、目录、源码和接口都是要检查的事实，不能虚构。
2. **先建基线再改造：**每个模块都要能与上一个稳定版本比较。
3. **一次处理一个可验收工作单元：**限制变更范围，保留工作区版本。
4. **测试与错误必须保留原始输出：**不编造成功、不通过改测试掩盖失败。
5. **每次修复必须解释证据：**检查真实报错，必要时回滚而不是无限重试。
6. **结果报告与计划严格分开：**已完成、已执行未验证、失败、阻塞和未执行分别列出。
7. **权限与副作用受控：**未经确认不执行危险动作，未知提交结果不得盲目重发。
8. **主动提出删减建议：**新增模块如无可复验收益，建议回退或移除。

第九部分：当前成果、尚未完成工作与风险

9.1 对话中已经完成的“文稿类成果”

- 形成十项 CUA 研究提案与各自的研究问题、实验思路和失败判据。
- 对用户提交的十项多智能体文献综述进行批判性评价，指出标题、证据与机制表述问题。
- 建立四种多智能体对照架构和分阶段科研实施思路。
- 在对话中曾产出《Linux CUA 十项科研论文与完整实施计划》PDF/Word，资料性质为研究提案及计划。

- 明确 DeepSeek Harness + DeepSeek API 的系统分层，以及“不改官方核心、先核实扩展点”的约束。
- 形成两版较长的针对性研究文稿，后续将研发重心转向高质量自主编程。
- 在用户指出最初“边写边测”说明敷衍后，完成一份更详细的自主软件工程实施规范，包括九级能力、七层测试、失败状态机和执行指令。

9.2 没有证据表明已经完成的工作

本次对话没有展示对当前 DeepSeek Harness 仓库的真实读取与 SHA 审核、对当前官方扩展 Schema 的逐项检查、Linux 真机 CUA 能力复现、首个产品独立验收、四组或六组对照实验执行、1,440 次运行结果、统计显著性分析，或多智能体集群已在用户机器落地的证据。因此应全部视为**待执行/待核实**，不能写成“已经构建成功”。

9.3 重大风险与对应措施

风险	对话形成的应对思路
模型推理能力不足	区分模型错误与执行错误，考虑模型/检索/任务边界
Harness 扩展能力不足	核实版本、接口和外围服务可行性，禁止凭空写 diff
GUI 失焦与不同 Linux 桌面不兼容	记录 X11/Wayland、权限和窗口状态，按场景选工具
自动修复越改越坏	最小补丁、重复失败检测、回归与回滚
自编测试掩盖缺陷	独立验收、测试变更审计、隐藏测试隔离
长任务崩溃导致重复副作用	持久化状态、幂等核验、未知时暂停
记忆变多但内容过时	环境指纹、版本匹配、过期与回归验证
蜂群耗费成本却无收益	等预算对照、工作区隔离、选择性并行或移除
文献库庞大却没有产品	先通过首个真实交付门槛，再扩大研究范围

第十部分：最终共识与最小下一步行动

10.1 可以概括为一句话的项目方向

以 DeepSeek Harness + DeepSeek API 为底座，优先做出一个能在 Linux 上真实理解项目、增量编程、边写边测、修复报错并独立交付软件的单 Agent；随后用可复现对照实验判断规划、记忆、恢复和蜂群等改装是否值得保留。

10.2 首轮行动清单

- ☐ 确认 Harness 当前版本、来源、SHA 和实际支持的扩展接口。
- ☐ 确认 DeepSeek API 的真实调用、模型设置与费用统计方式。
- ☐ 确认 Linux 环境、工具权限、项目工作区与已有改动。
- ☐ 录制或保存“读文件—改文件—运行命令—读报错”的真实执行证据。
- ☐ 为首个小型软件写清独立验收契约，不让开发 Agent 随意修改最终标准。
- ☐ 建立差异检查、测试日志和三态/多态完成状态。
- ☐ 在一个小工作单元上验证“编写—测试—诊断—修复—回归”流程。

□ 实际运行软件，关闭重启并读回持久化数据。
□ 将失败与瓶颈分门别类记录，再决定下一项值得改装的模块。
□ 只在单 Agent 基线足够明确后启动多 Agent 对照实验。

10.3 完成后应提交的证据包

至少包含当前环境与版本记录、任务与验收标准、代码修改差异、真实工具执行日志、测试记录、运行与重启证据、独立验收结论、失败与 Pending 清单、模型费用/耗时，以及下一阶段候选改装的价值判断。整理完成时应让开发 Agent **解释对文献与架构的理解、指出不可靠假设并给出建议**，而不仅仅输出“整理完成”。

附录 A：本次对话涉及的公开文献与项目入口

以下是对话中出现的重要公开入口，作为追溯与后续查新的线索；**本次总结未联网核实 2026 年最新提交、许可或最新实验结果**。

1. CAMEL —https://arxiv.org/abs/2303.17760
2. AutoGen —https://arxiv.org/abs/2308.08155
3. MetaGPT —https://arxiv.org/abs/2308.00352
4. ChatDev —https://arxiv.org/abs/2307.07924
5. Reflexion —https://arxiv.org/abs/2303.11366
6. Multi-Agent Debate —https://arxiv.org/abs/2305.14325
7. GPTSwarm —https://arxiv.org/abs/2402.16823
8. AgentVerse —https://arxiv.org/abs/2308.10848
9. AgentBench —https://arxiv.org/abs/2308.03688
10. OSWorld —https://arxiv.org/abs/2404.07972
11. SWE-bench —https://github.com/SWE-bench/SWE-bench
12. SWE-agent —https://github.com/SWE-agent/SWE-agent
13. OpenHands —https://github.com/All-Hands-AI/OpenHands
14. Aider —https://github.com/Aider-AI/aider
15. BrowserGym —https://github.com/ServiceNow/BrowserGym
16. Microsoft UFO —https://github.com/microsoft/UFO
17. AutoGen 源码入口—https://github.com/microsoft/autogen
18. Temporal —https://docs.temporal.io/
19. Model Context Protocol —https://modelcontextprotocol.io/
20. OSWorld 源码入口—https://github.com/xlang-ai/OSWorld

附录 B：本次对话中关键概念的统一解释

CUA（Computer-Use Agent）：观察计算机环境并执行实际桌面、浏览器、文件或终端操作的智能体；可以使用不同工具组合，不必全程 GUI 点击。

Harness：运行模型—工具交互的基础协调框架；具体功能取决于真实版本。

自主软件工程闭环：把需求理解、代码修改、真实执行、测试、根因分析、修复与交付连成有证据的循环。

独立验收：不完全由开发 Agent 自己决定合格标准，使用真实产物和外部测试检查用户目标。

回归测试：确保新修改没有破坏已有功能。

工作区隔离：不同 Agent 使用独立代码状态或执行环境，避免并发写入冲突。

幂等：同一操作重复执行不会造成额外效果；真实外部系统未必具备这一性质。

消融实验：移除某个机制，看是否真的影响结果，从而判断它的独立贡献。

负迁移：旧经验或某种机制在新任务上反而使表现下降。

Pending / Unknown：尚无充分真实证据，不等于完成。

总结结语这次对话并没有得出“DeepSeek Harness 经改装就一定变得强大”这样的结论。它最终确立了一条更审慎、可操作的研发路线：**以用户可用的软件成果为目的，以真实测试与独立验收为标准，以失败和成本证据为依据，逐步判断每一项改装是否值得长期保留**。